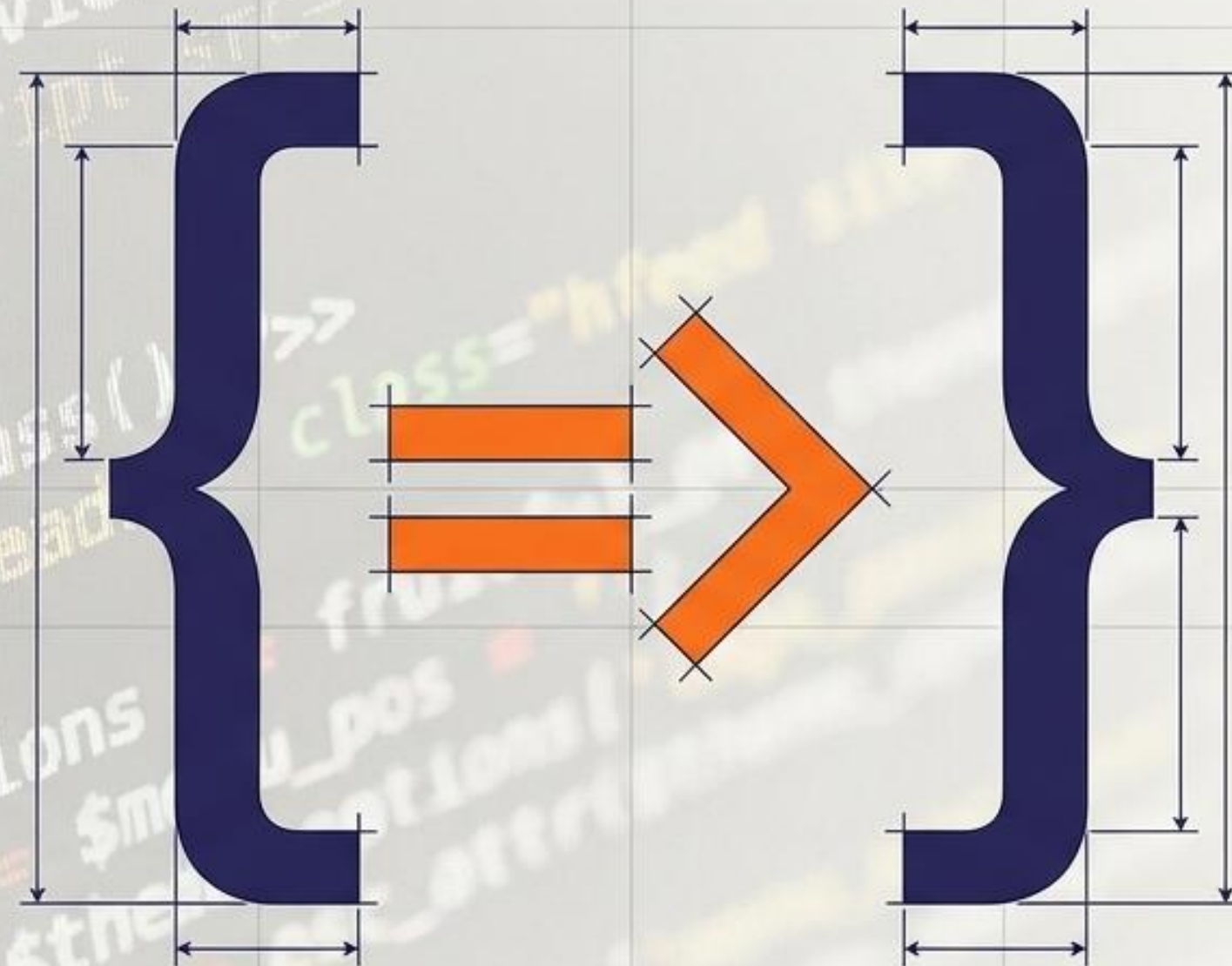




JavaScript Moderno (ES6+) e Listas Dinâmicas

Dando vida aos dados na web.

Programação Front-end 2 (PFE2) – Aula 02

Atualização de Sistema: JavaScript Moderno e Listas Dinâmicas

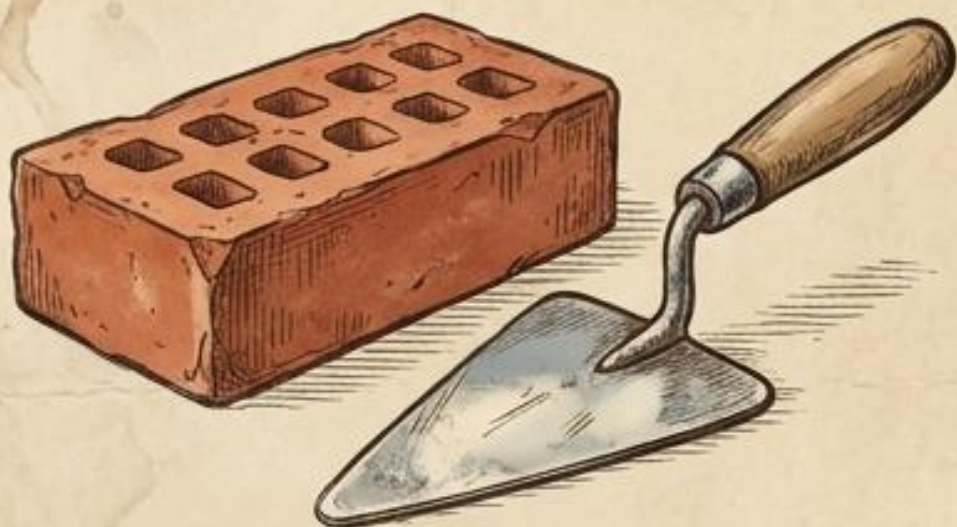
Semana 02 | PFE2 - Técnico em
Desenvolvimento de Sistemas

Substituindo ferramentas manuais
por automação de alto nível.



O Dilema do Prédio de 100 Andares

Manual Era



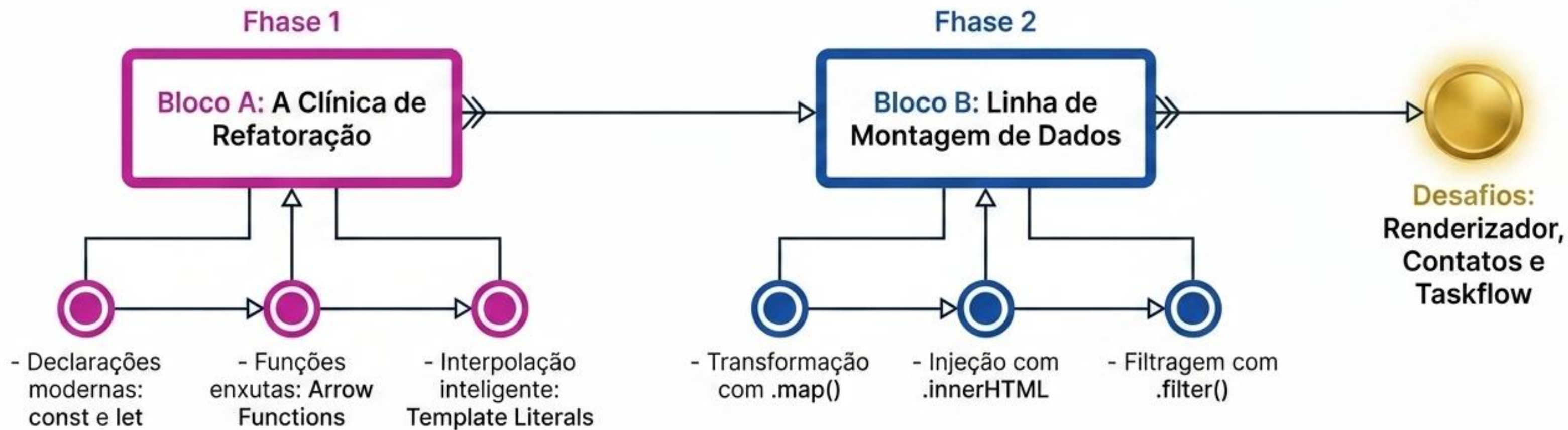
```
<script>
  var element = document.getElementById('my-brick');
  element.style.display = 'block';
  element.innerHTML = 'Brick 1';
  var brick2 = document.createElement('div');
  brick2.className = 'brick';
  document.body.appendChild(brick2);
</script>
```

Construir um arranha-céu tijolo por tijolo manipulando o DOM manualmente é inviável. Hoje, abrimos a caixa de ferramentas elétricas do desenvolvedor.

Automation Era



Roteiro de Evolução: A Oficina de Hoje



Laboratório 1: Limpando a Casa (Guiado)

Antes (Com Dor)

1. Onde podemos trocar let/var por const?

Dica: Variáveis que selecionam elementos do DOM raramente mudam de valor.

2. Como reescrevemos os callbacks?

Agora (Sem Dor)

Transforme todos os **addEventListener** para utilizarem Arrow Functions de forma concisa.

Bloco A: Prática Guiada de Refatoração

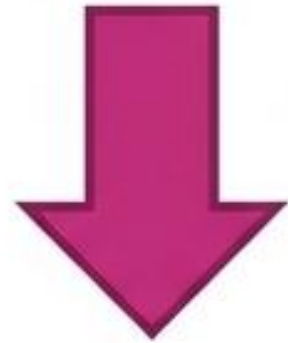
> O código legado não está errado, mas é ineficiente.

> O JavaScript Moderno (ES6+) permite escrever código mais seguro, limpo e previsível.

> INICIANDO ATUALIZAÇÃO DE SINTAXE...

Ferramenta 1: Controle Rigoroso de Variáveis

```
var nomeUsuario = 'João';
```



```
const  nomeUsuario = 'João';
```

Abandonamos o **var**. Agora o **escopo** é **previsível**.

Use **const** por padrão (valor imutável).

Use **let** apenas quando o valor precisar mudar.

Arrow Functions: Ordens Diretas

É uma **forma mais direta** de dar uma ordem. Em vez de "Por favor, execute a seguinte função...", é simplesmente "**Faça isto =>**".

```
function(a, b) { return a + b }
```



```
const somar = (a, b) => a + b;
```


Ferramenta 2: Arrow Functions

```
botao.addEventListener('click', function() {  
  // lógica  
});
```



```
botao.addEventListener('click', () => {  
  // lógica  
});
```

- Sintaxe mais concisa e comportamento inteligente.
- Perfeito para callbacks.
- O símbolo => substitui a palavra-chave function.

Manipulação de Strings sem Dor

Antes (Com Dor)

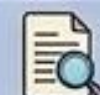


Agora (Sem Dor)

``${Olá, nome!}``

 **Múltiplas Linhas:** Permite strings em várias linhas sem quebrar o código (usando crases `).

 **Interpolação:** Insere variáveis diretamente na string usando `${variavel}`.

 **Legibilidade:** Torna o código muito mais limpo para ler e manter.

Ferramenta 3: Template Literals

```
var saudacao = 'Bem-vindo, ' +  
nomeUsuario + '!';
```



```
const saudacao = `Bem-vindo, ${nomeUsuario}!`;
```

- Chega de quebrar strings e somar variáveis com **+**.
- Use **crases** para criar strings dinâmicas.
- Injete variáveis diretamente com **\${ }**.

O Novo Padrão de Variáveis

Característica	var (Evitar)	 let (Usar quando mudar)	 const (Preferir sempre)
Uso	Sim	Sim	Não
Reatribuição	Alto	Sim	Não
Risco de Sobreescrita	Alto	Baixo	Zero

Regra de ouro: Use const sempre. Só mude para let se tiver certeza absoluta de que o valor precisa mudar durante a execução.

Matriz de Atualização de Sintaxe: O Antes e o Depois

Ferramenta Manual (ES5)	Ferramenta Elétrica (ES6+)
Declaração: var (Escopo confuso)	Declaração: const / let (Escopo de bloco, seguro)
Função: function() {} (Sintaxe longa)	Função: () => {} (Conciso, escopo léxico)
Texto: "Oi, " + nome + "!" (Concatenação frágil)	Texto: `Oi, \${nome}!` (Interpolação legível)

Bloco B: A Linha de Montagem de Dados

O objetivo é transformar dados **puros** (Objetos JS) em **elementos visuais** (HTML).

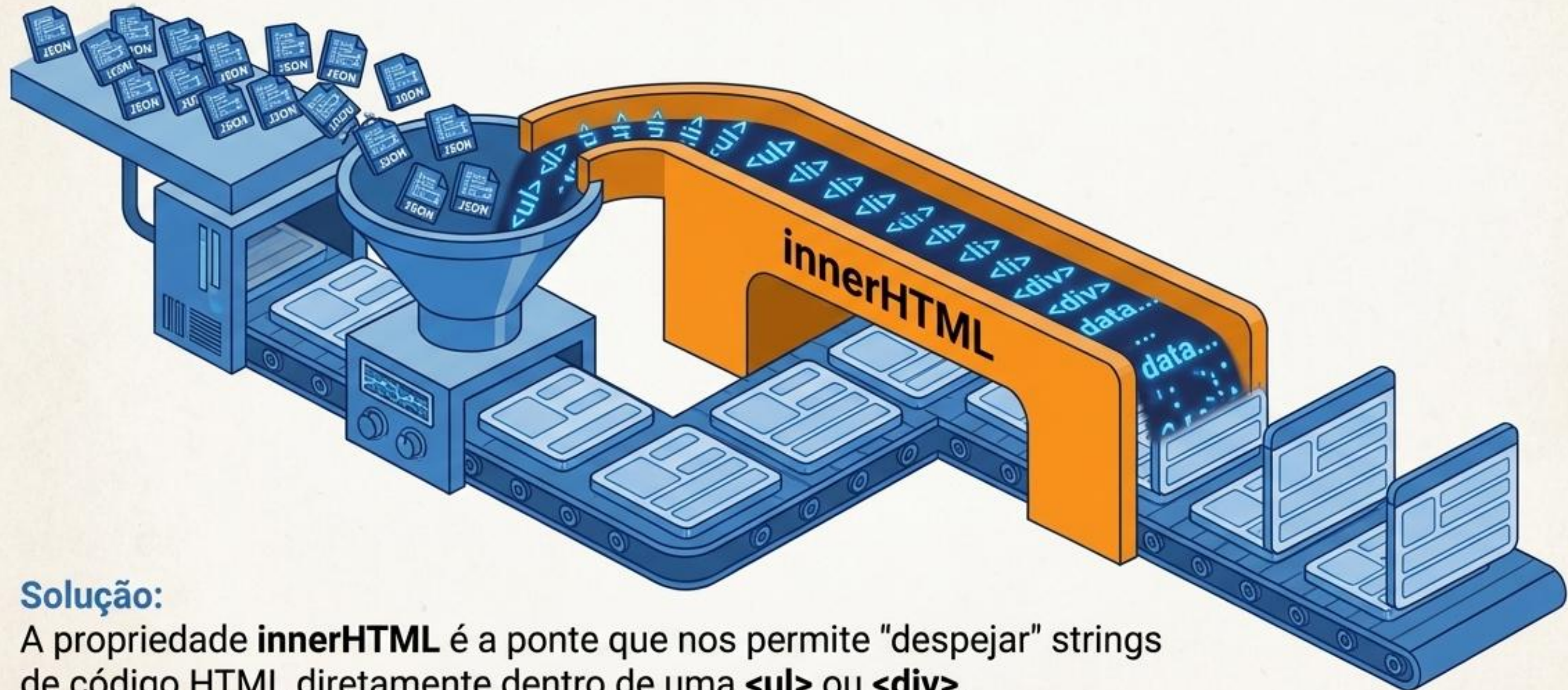
O **Método .map()**: Uma máquina que passa por cada item, aplica uma regra, e devolve uma nova lista transformada.



```
const sumos = uvas.map((uva) => transformarEmSumo(uva));
```


O Desafio de Renderizar Listas

Recebemos **milhares** de dados de uma API, mas eles estão no formato '**JavaScript**'. O navegador só entende HTML. **Como exibir 1.000 itens na tela em milissegundos?**



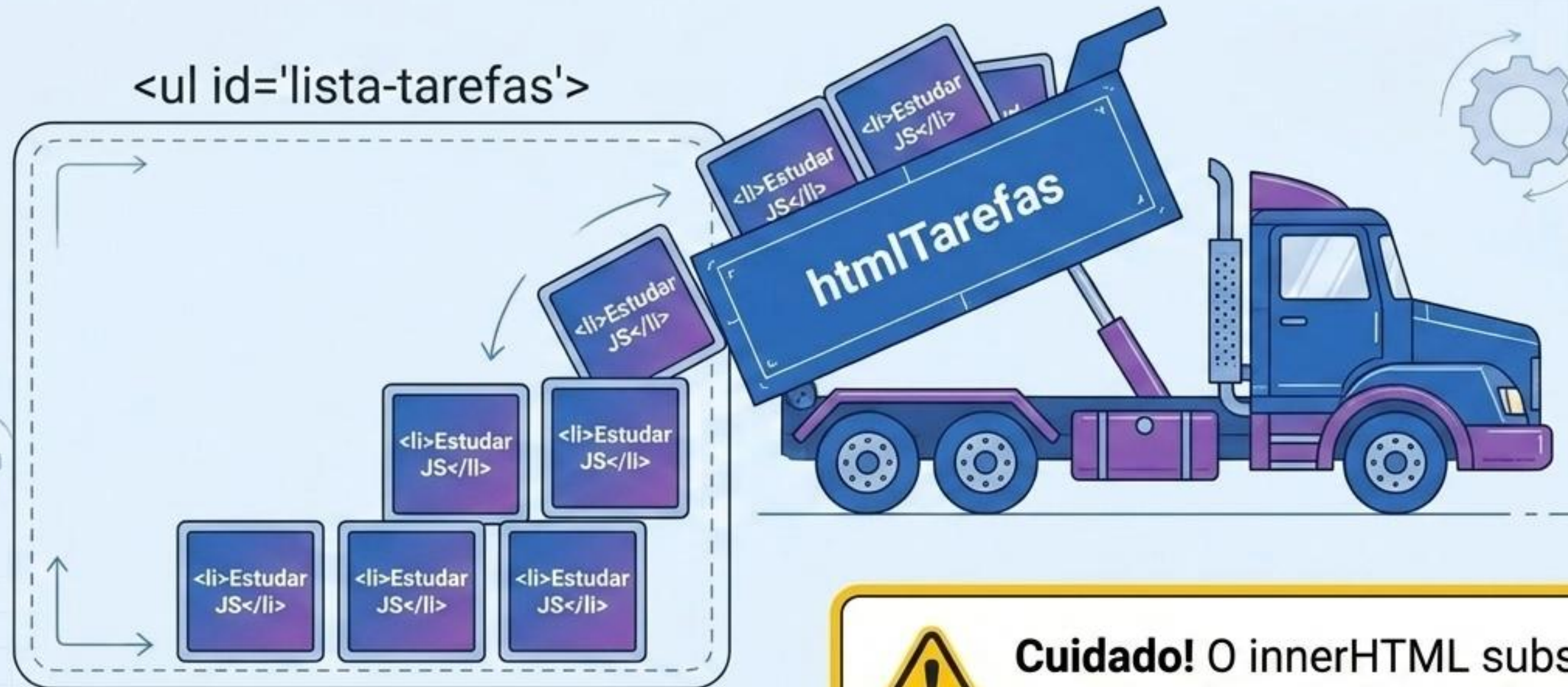
Solução:

A propriedade **innerHTML** é a ponte que nos permite "despejar" strings de código HTML diretamente dentro de uma `<ul>` ou `<div>`.

Despejando o Resultado: innerHTML

Após gerar as strings HTML, precisamos injetá-las na página.

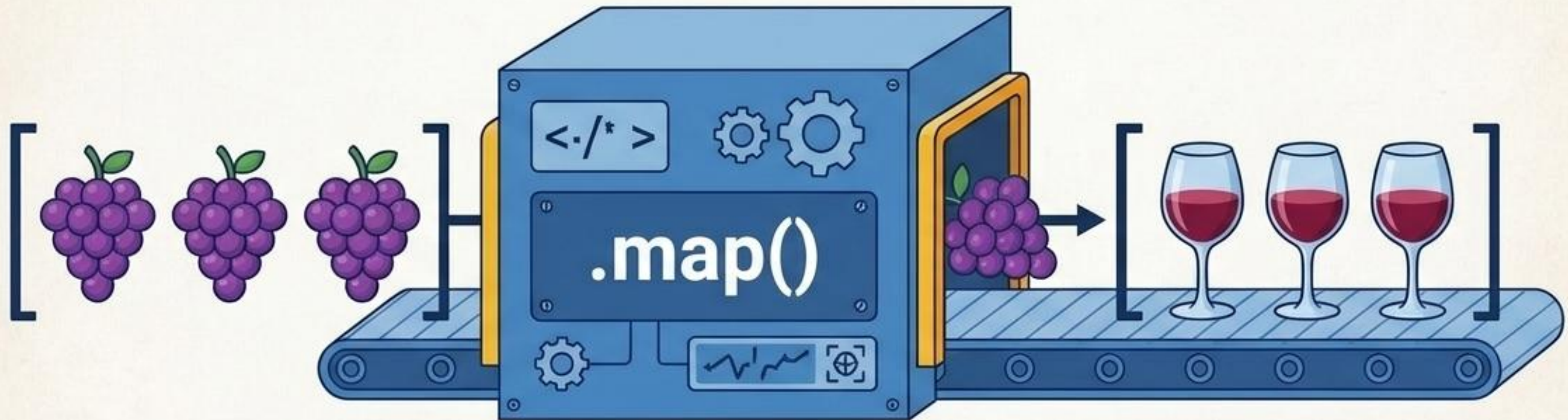
A propriedade **.innerHTML** permite pegar uma string e "despejá-la" dentro de um elemento no DOM.



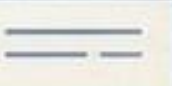
Cuidado! O `innerHTML` substitui todo o conteúdo anterior do elemento.

A Máquina de Transformação: .map()

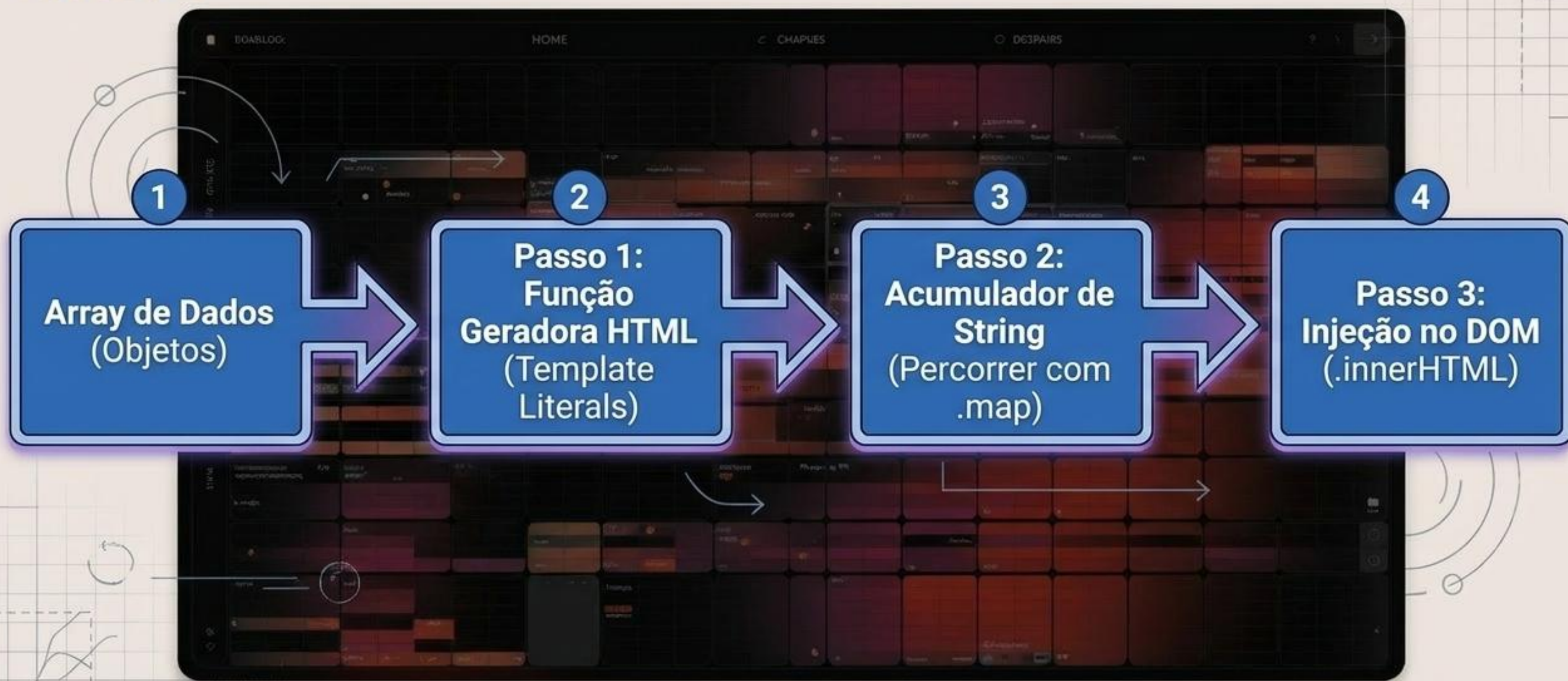
Passa por **cada item** da lista, aplica uma transformação, e devolve um novo array com exatamente o **mesmo tamanho**.



```
const descricoes = tarefas.map((tarefa) => {  
  return tarefa.descricao;  
});
```



Atividade 1 Guiada: Renderizador de Lista de Tarefas



A Lógica em Ação (Solução Guiada)

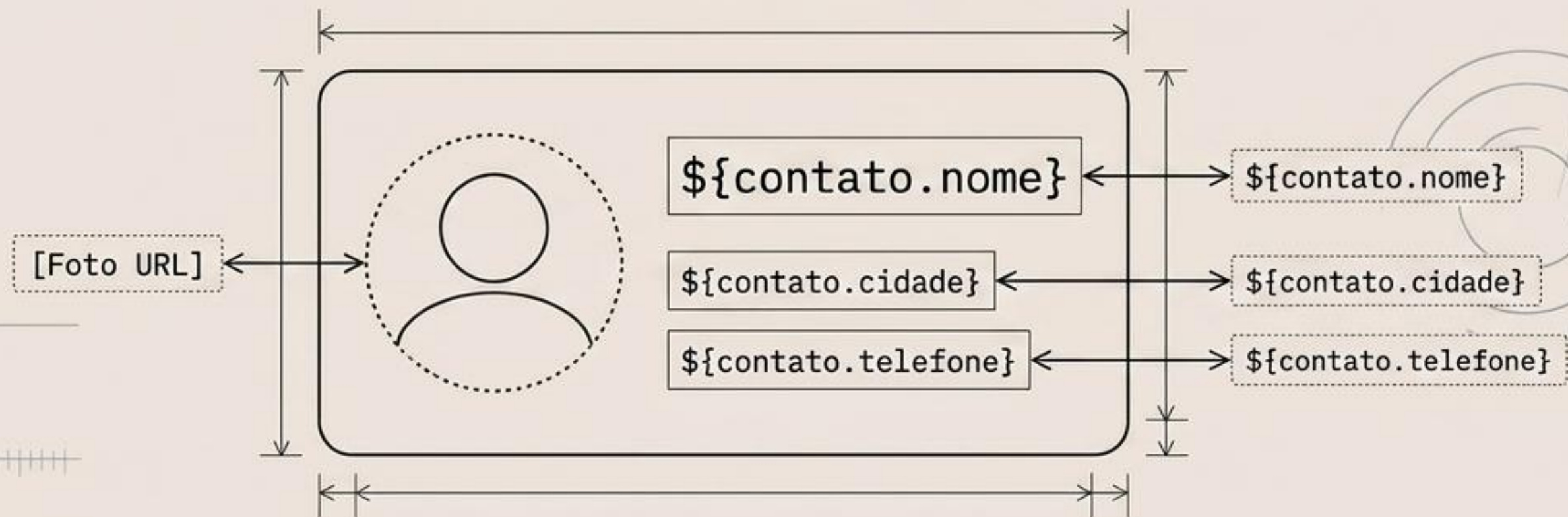
Criando o bloco HTML com Template Literals e injetando no DOM.

```
const listaTarefasUL = document.querySelector('#lista-tarefas');

const criaItemTarefa = (tarefa) => {
  const status = tarefa.concluida ? 'Concluída' : 'Pendente';
  return `
    <li>
      <h3>${tarefa.descricao}</h3>
      <p>Status: ${status}</p>
    </li>
  `;
};
```


Atividade 2 Individual: Lista de Contatos Dinâmica

- Use os mesmos padrões, mas em um novo contexto.
- **Objetivo:** Ler um array de Contatos.
- **Meta:** Renderizar cards na tela com `.map()` e `.innerHTML`.
- Preparação crucial para consumir APIs na próxima semana.

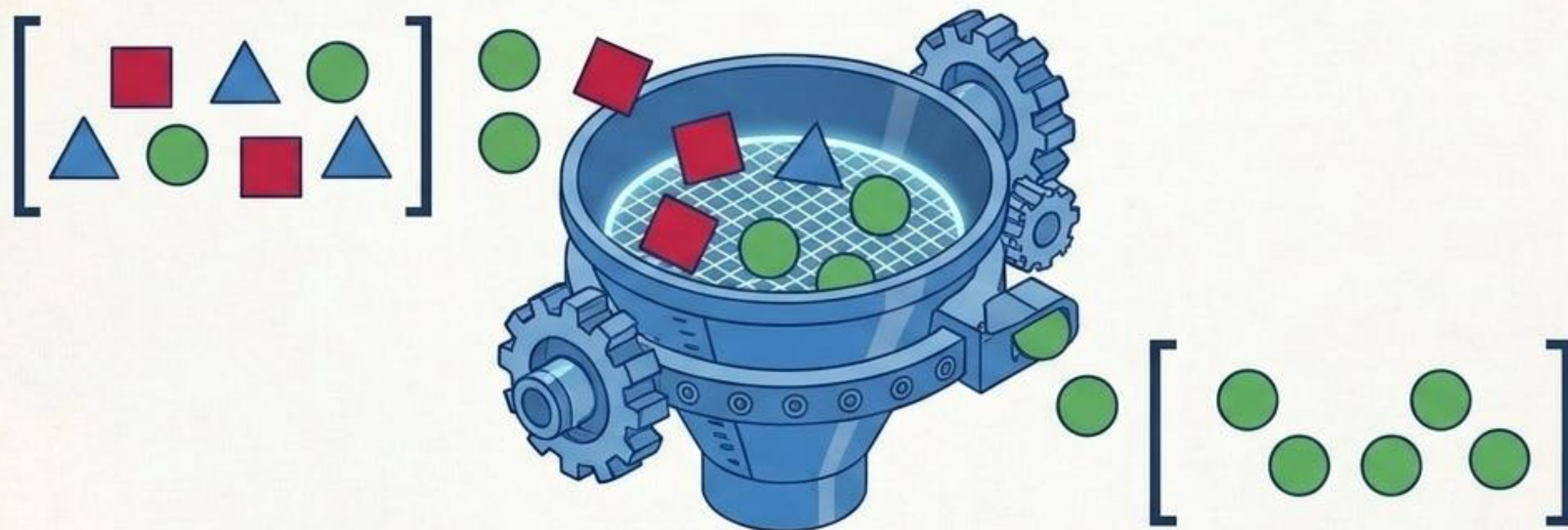


Requisitos da Entrega

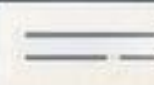
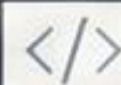
- [] Utilizar o arquivo script.js base fornecido na atividade.
- [] Criar uma função auxiliar utilizando a sintaxe de **Arrow Function** =>.
- [] Interpolar os dados (nome, cidade, telefone, foto) na estrutura HTML utilizando **Template Literals** `\${}`.
- [] Injetar a string resultante na **** principal da página.

A Peneira de Condições: `.filter()`

Testa cada **item** com uma condição. Devolve um novo array apenas com os itens que **passaram no teste** (onde a condição é verdadeira).

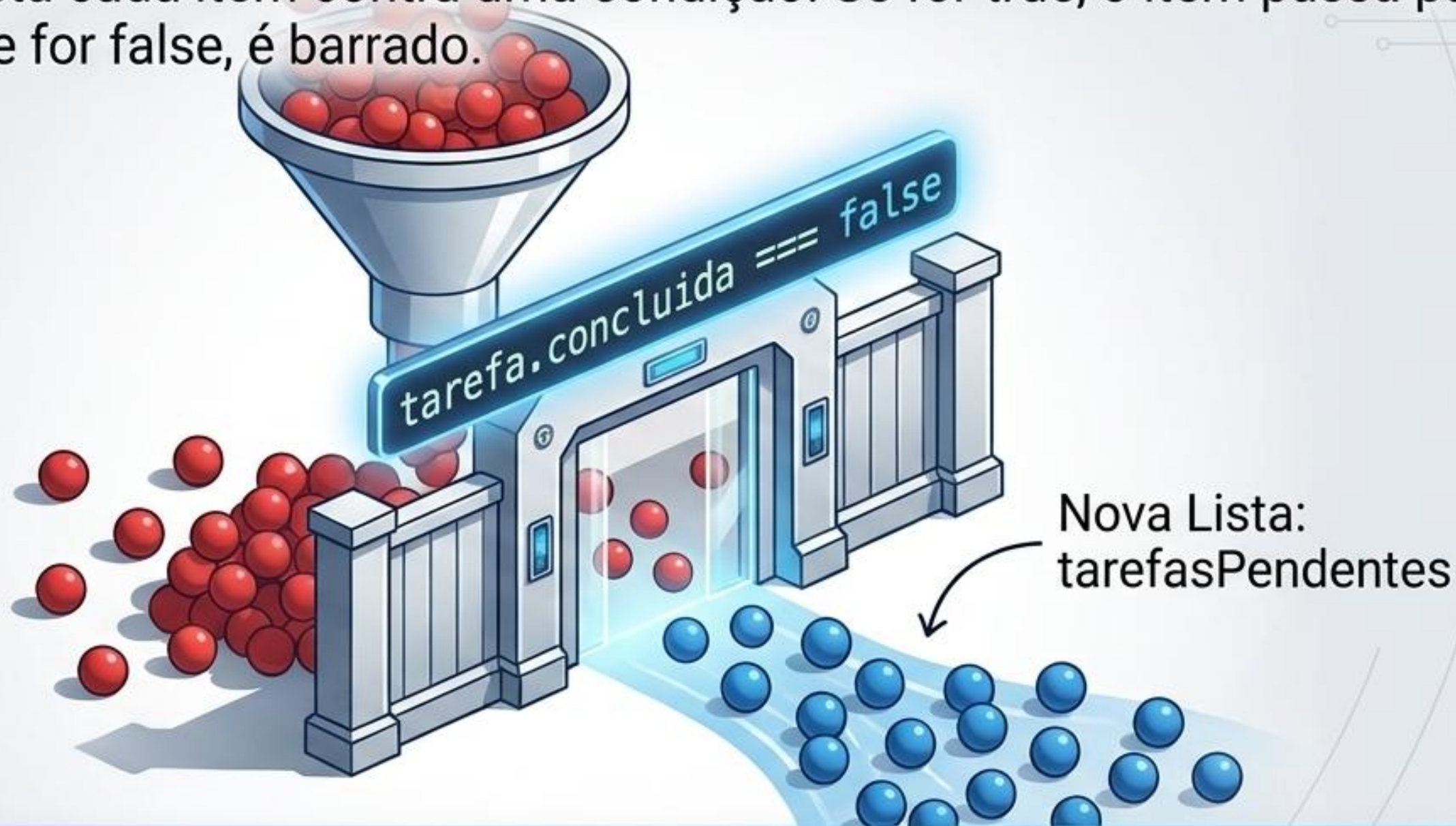


```
const pendentes = tarefas.filter((tarefa) => {  
  return tarefa.concluida === false;  
});
```



Pílula de Aprendizado: O Guardião de Dados (.filter)

O que faz? Testa cada item contra uma condição. Se for true, o item passa para a nova lista. Se for false, é barrado.



```
const pendentes = tarefas.filter(task => task.completed === false);
```


O Seu Momento de Codificar

Chegou a **hora de plugar as ferramentas elétricas na tomada**. Teremos dois desafios progressivos de manipulação de dados e renderização no DOM.

O instrutor assumirá agora o papel de 'Gerente de Projeto' para guiar a sua execução.

Atividade 3 em Duplas: Taskflow com Filtros

Desafio: Adicionar botões e usar `.filter()` para re-renderizar a lista ao clicar.

Taskflow

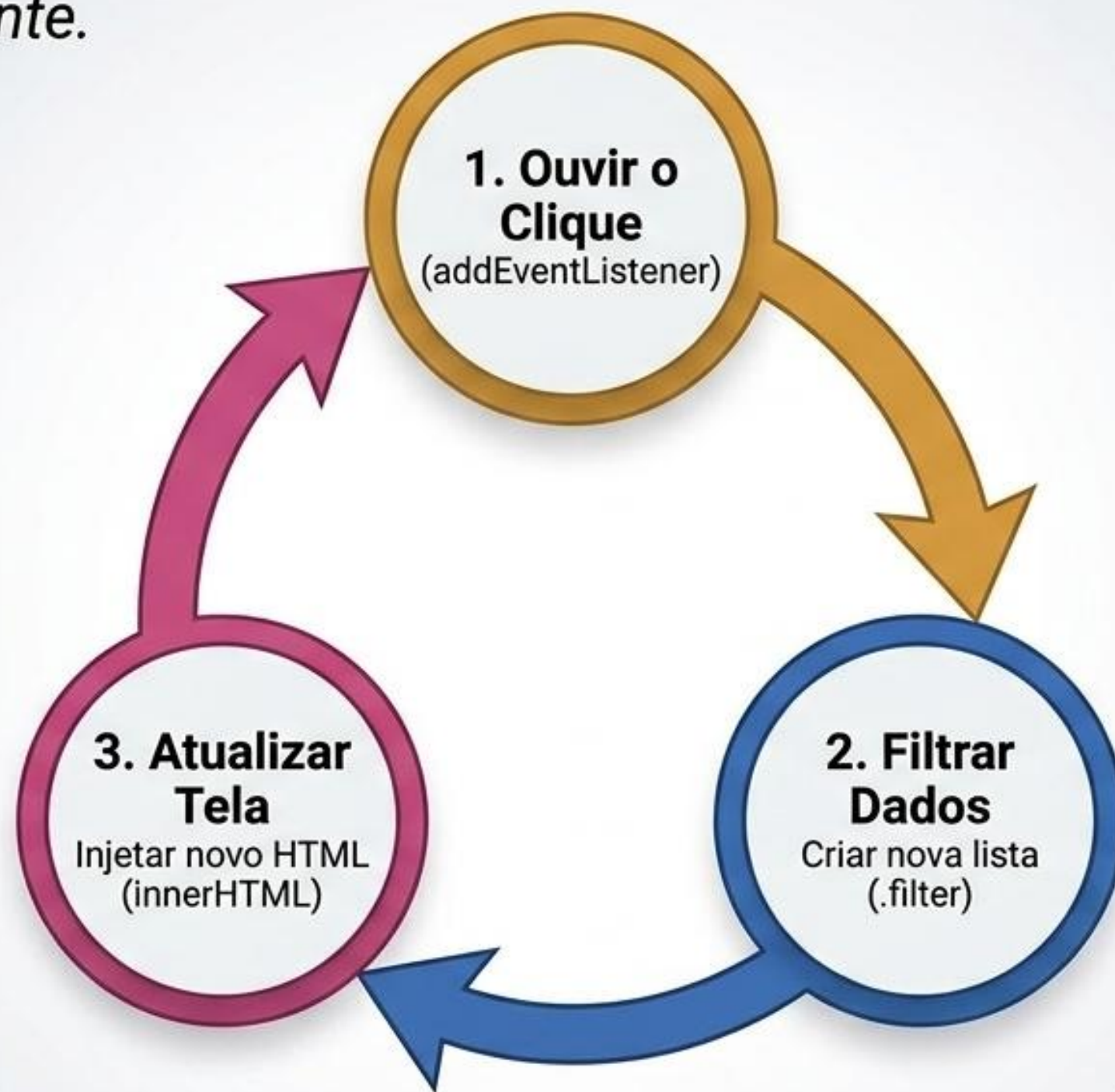
[Mostrar Todas]

[Mostrar Pendentes]

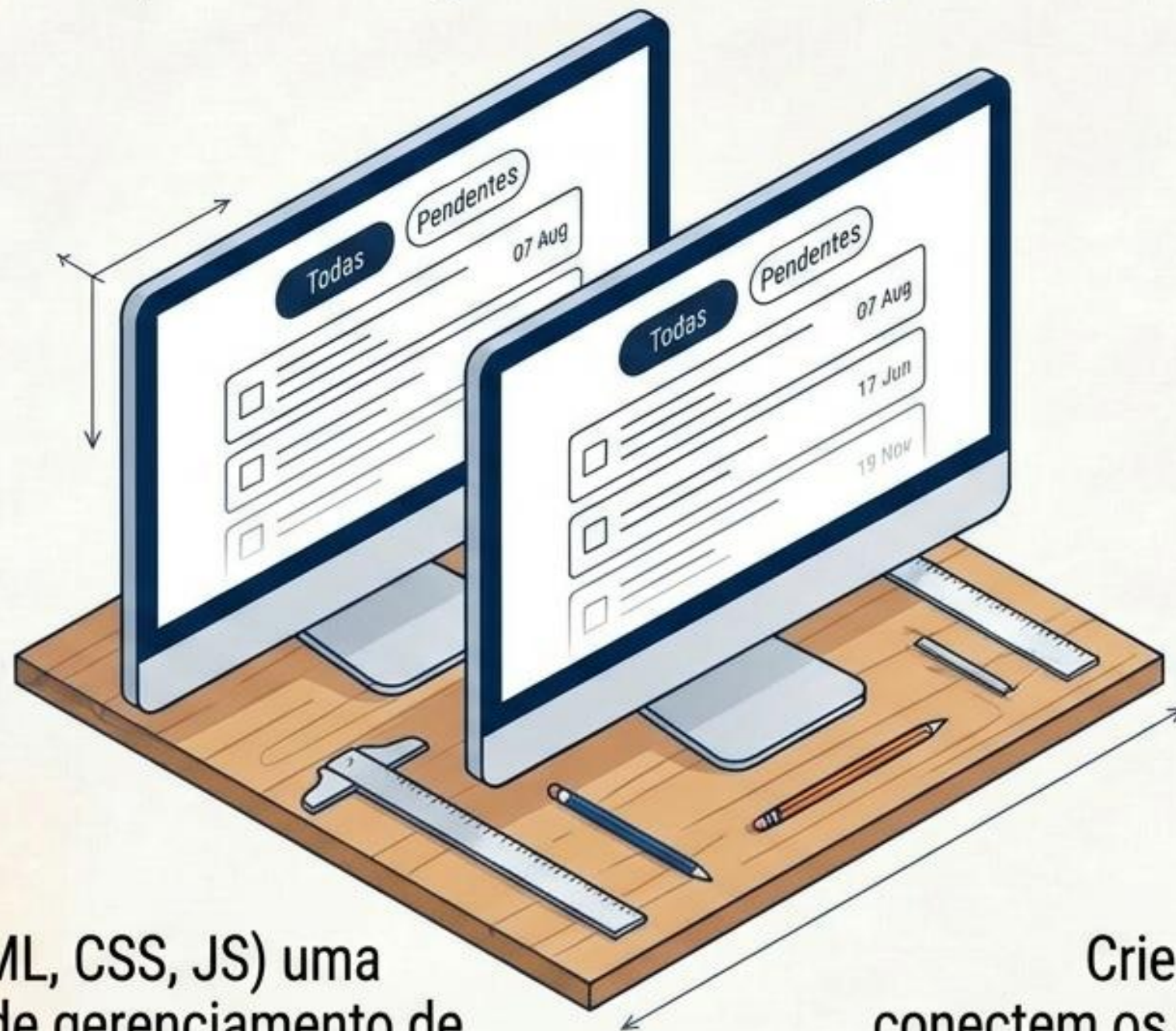
```
btnPendentes.addEventListener('click', () => {  
  const pendentes = tarefas.filter(...);  
  renderizar(pendentes);  
});
```


O Segredo da Interatividade Moderna

Nota mental: Hoje apagamos e reescrevemos o DOM manualmente. Em breve, o React fará isso automaticamente.



Desafio 3 (Em Duplas): Projeto Taskflow



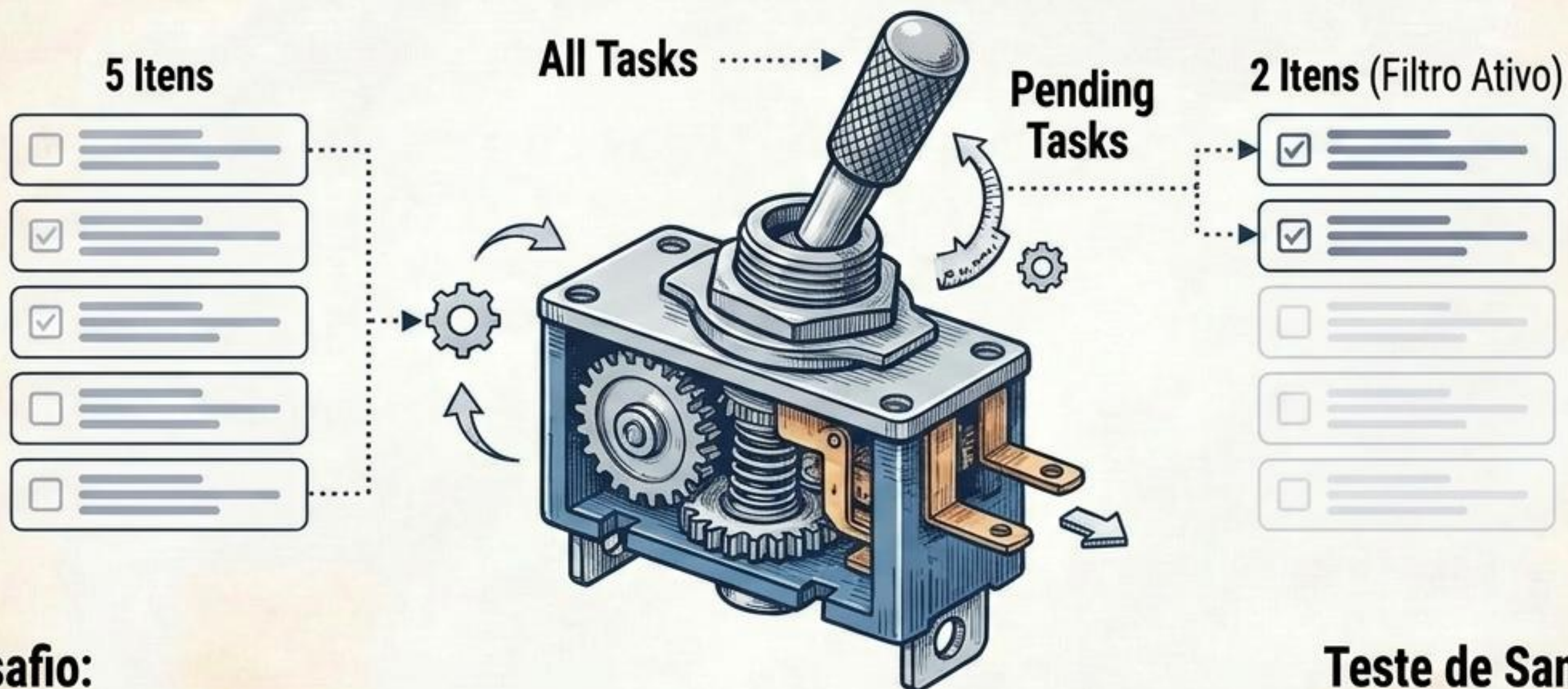
Objetivo:

Construir do zero (HTML, CSS, JS) uma interface minimalista de gerenciamento de tarefas com filtragem dinâmica de dados.

Setup Inicial:

Criem a estrutura visual e conectem os seus scripts antes de iniciar a lógica de dados.

Dominando a Lógica de Filtragem



O Desafio:

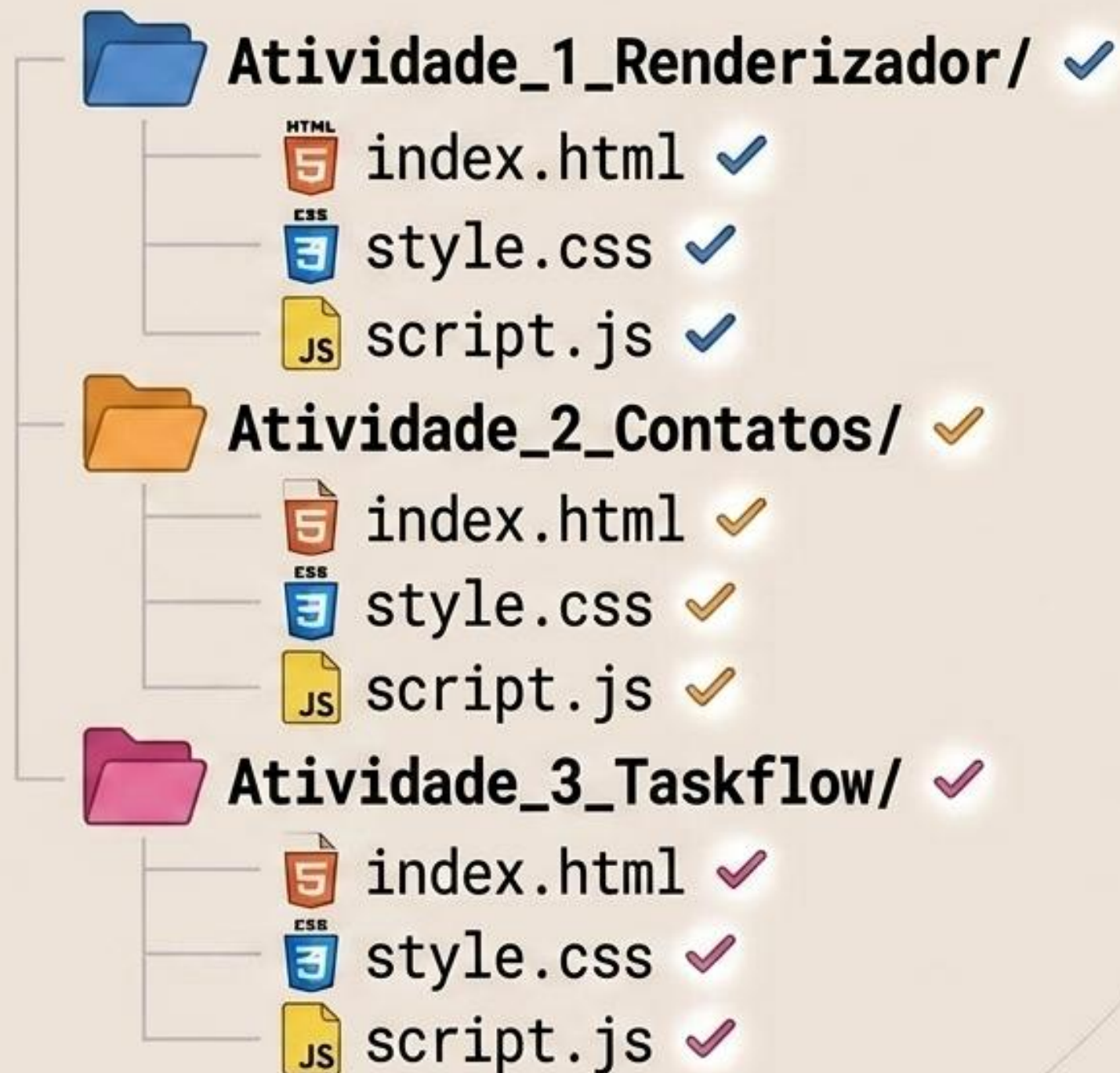
Utilizar o método `.filter()` para criar um novo array que contenha apenas as tarefas que possuem a propriedade `concluida: false`.

Teste de Sanidade:

Antes de enviar para o HTML, exiba o array resultante no `console.log()` para garantir que a sua 'peneira' funcionou corretamente.

Checklist de Entrega: Seu Repositório Final

Organize o seu workspace. Você deve entregar 9 arquivos em 3 pastas.

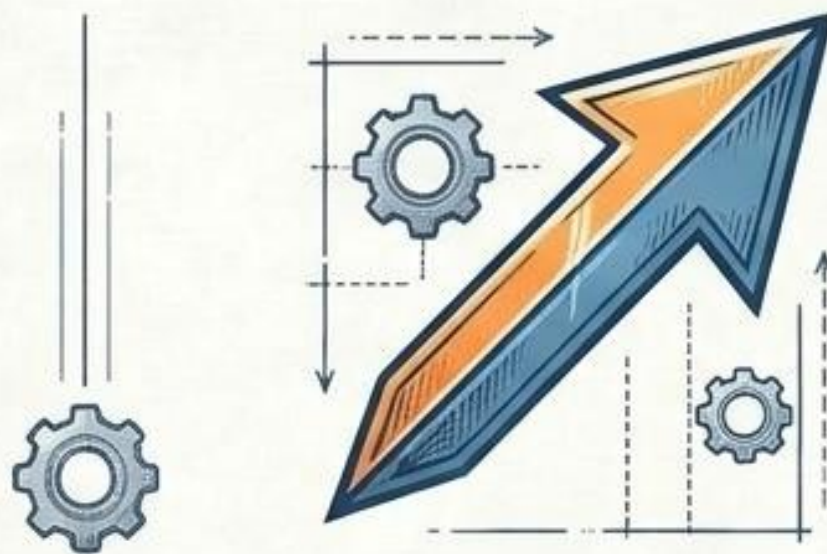


O Seu Novo Arsenal Front-end



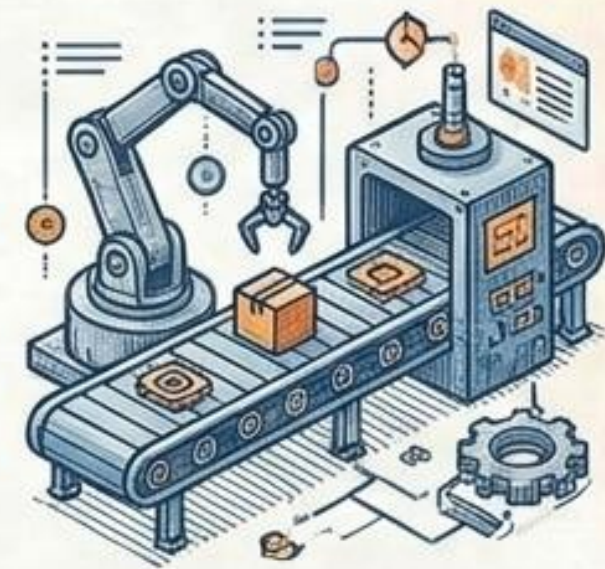
Variáveis Seguras:

Substituímos o velho `var` pela previsibilidade do `let` e `const`.



Sintaxe Limpa:

Adotamos Arrow Functions e Template Literals para um código direto.



Manipulação Inteligente:

Descobrimos como processar grandes volumes de dados usando `.map()` e `.filter()`.

Preparem-se, pois estas ferramentas elétricas são a fundação absoluta para a introdução ao React!

Folha de Referência: O Seu Novo Arsenal (ES6+)

Tire um print! Estas são as ferramentas diárias de um Front-End.

`const / let`

Variáveis blindadas e com escopo contido.

`() => {}`

Arrow Functions para declarações diretas e limpas.

`` ${ } ``

Template Literals para misturar HTML e Variáveis.

`.map()`

A máquina que transforma listas inteiras.

`.filter()`



O portão de segurança que seleciona dados específicos.

`.innerHTML`

A ferramenta de injeção que joga HTML na página.